LEXICON

NOSQL & MONGODB

Designing documents

Embed or reference, schema validation, and the aggregation pipeline

LEXICON

Today

- After today you can:
  - Decide when to embed data and when to reference it
  - Explain why copied data is sometimes good and sometimes a problem
  - Add rules to a collection with schema validation
  - Summarise data with the aggregation pipeline: $match, $group, $sort, $project
  - Use $unwind and $lookup

LEXICON

Recap

- Operators: $gt, $lt, $in, $ne... written as {"field": {"$op": value}}
- "details.color" reaches inside objects. {"tags": "winter"} searches in arrays
- find(filter, projection).sort(...).limit(...)
- update_one / update_many with $set, $inc, $push, $pull, $addToSet
- Empty filter = everything. No undo: check with find first!

LEXICON

Quick questions

1. Write a filter for products that cost between 200 and 600 kr.
2. What is the difference between $push and $addToSet?
3. Why should you run find() before update_many() or delete_many()?

LEXICON

Design starts with questions

- SQL: normalise first, every fact in one place, JOIN when you read
- MongoDB: first ask 'how will the app read the data?'
- Rule of thumb: data that is read together is stored together

LEXICON

Embed or reference?

- Embed = put it inside the document
- Reference = store the _id of another document, like a foreign key

```
# Embed: the items live inside the order
{"_id": 1, "items": [{"name": "Hoodie Black", "quantity": 1}]}

# Reference: the order points to the customer
{"_id": 1, "customer_id": 1}
```

LEXICON

When to embed, when to reference

Embed when...	Reference when...
It's read together with the parent	It's often read on its own
It belongs to one parent only	Many documents share it
There are a few, not thousands	The list can grow without limit
It rarely changes after it's written	It changes often

LEXICON

Our webshop design

Data	Where	Why
Order items	Embedded in the order	Always shown with the order, never change
Customer	Referenced: customer_id	One customer has many orders, details change
Products	Own collection	Shown on their own, stock changes all the time
Product name and price in items	Copied into the order	The receipt must show what the customer paid

LEXICON

Copies have a price

- Copying data (denormalisation) makes reading fast
- Good copies: things that should freeze in time (price on a receipt)
- Bad copies: things that must stay the same everywhere (a customer's email)
- If you copy the email into 1000 orders, a change means 1000 updates

LEXICON

Watch out: arrays that grow forever

- A document can be at most 16 MB
- Big documents are slow to read and update
- Reference instead: orders have a customer_id

```
# Bad idea: every order inside the customer
{"_id": 1, "first_name": "Anna",
 "orders": [ {...}, {...}, {...}, ... thousands ... ]}
```

LEXICON

Rules for your documents

- Schema validation: MongoDB's version of NOT NULL and CHECK
- Documents that break the rules are rejected

```
db.create_collection("reviews", validator={"$jsonSchema": {
    "required": ["product_id", "rating"],
    "properties": {
        "product_id": {"bsonType": "int"},
        "rating": {"bsonType": "int", "minimum": 1, "maximum": 5}}}})
```

LEXICON

Aggregation: an assembly line

- A pipeline is a list of stages
- Each stage gets the documents from the stage before

```
db.orders.aggregate([
    {"$match": {"status": "delivered"}},    # step 1
    {"$group": {"_id": "$customer_id",      # step 2
                "orders": {"$sum": 1}}},
    {"$sort": {"orders": -1}},              # step 3
])
```

LEXICON

Stages and their SQL friends

Stage	SQL friend	Does
$match	WHERE / HAVING	Keep documents that match a filter
$group	GROUP BY	Make piles and count, sum, average
$sort	ORDER BY	Sort
$project	SELECT	Choose and calculate fields
$limit	LIMIT	Keep the first N
$unwind	(none)	One document per array item
$lookup	JOIN	Fetch matching documents from another collection

LEXICON

$group

- "$price" with a dollar = the value of the field price
- Accumulators: $sum, $avg, $min, $max, $push

```
{"$group": {
    "_id": "$category",                 # what to group by
    "products": {"$sum": 1},            # count
    "avg_price": {"$avg": "$price"}     # average
}}
# SELECT category, COUNT(*), AVG(price) FROM products GROUP BY category
```

LEXICON

$unwind: one document per array item

```
# Before: 1 document
{"_id": 1, "items": [{"name": "Hoodie Black"}, {"name": "Cap Logo"}]}

# After {"$unwind": "$items"}: 2 documents
{"_id": 1, "items": {"name": "Hoodie Black"}}
{"_id": 1, "items": {"name": "Cap Logo"}}
```

LEXICON

$lookup: a JOIN in MongoDB

- The matches are put in a new array
- Use it when you need it, but design so you rarely need it

```
{"$lookup": {
    "from": "customers",           # the other collection
    "localField": "customer_id",   # field in orders
    "foreignField": "_id",         # field in customers
    "as": "customer"               # name of the new array
}}
```

LEXICON

Remember

- Design by asking how the app reads the data
- Embed what belongs together, reference what is shared or grows
- Copies are fast, but must be updated everywhere (unless they should freeze)
- Schema validation = MongoDB's NOT NULL and CHECK
- Aggregation: $match, $group, $sort, $project, $unwind, $lookup

LEXICON

This afternoon: lab

Open the exercise PDF. Work at your own pace, and ask for help when you get stuck.

LEXICON

Coming up tomorrow

Tomorrow: indexes, safe Python code, a small shop app, and a practice lab.